

Preguntas frecuentes

- Diferencia entre bug y defecto
- Diferencia entre Verificación y Validación
- Listas de Verificación y ejemplos
- Checklist (que es y cuando lo hago/para que)
- Especificación de Requisitos y ejemplos
- Cubrimientos por sentencia y por condición
- Árbol de complejidad ciclomática.
- Pruebas de regresión
- cuando hago una prueba de regresión
- Para que sirven los alfa y beta test, y cuales son los inconvenientes que tiene el alfa test.
- Que es el beta testing
 - * Porque se hace, cuales son sus beneficios
 - * Cual es el costo o dificultad que genera
- Ciclo de vida del defecto (para que sirve, por qué se usa)
- Métricas para qué sirve y decir tipos de métricas
- Smoke, para que se usa, un ejemplo
- Qué son las pruebas estáticas
- Fases de la pruebas dinámicas y definir preparación
- Bug Latente
- Complejidad Ciclomática
- Pruebas de Regresión
- Pruebas estáticas (fases)
- Que es un sistema (modelo V)
- Cubrimiento por Condicion y por Sentencia
- Tipos de Metricas

Resumen

- Si te pregunta que es un bug le decís "es la manifestación de un defecto" no empieces tratando de explicarle con tus palabras porque te mira torcido.

Diferencia entre defecto, bug y bug latente

- **Error:** equivocaciones humanas como los errores tipográficos o sintácticos.
- **Defectos:** son condiciones impropias de un programa que generalmente son el resultado de un error. No todos los errores producen defectos en el programa, como comentarios incorrectos o algunos errores de documentación.
 - Bug: es la manifestación de un defecto, ya sea durante la prueba o durante su uso (causa un comportamiento no deseado en el software)
 - Bug latente: Es un defecto que aún no se ha manifestado como un error en el comportamiento del sistema.
- **Fallas:** Es un mal funcionamiento en una instalación de usuario. Puede ser provocado por un bug, una instalación incorrecta, una falla del hardware
- **Problemas:** Son dificultades encontradas por los usuarios. Pueden provenir de fallas, mal uso o mal entendimiento.

Falla= Eventos relacionados al sistema

Problemas= Eventos relacionados a los humanos

Diferencia entre Verificación y Validación

- **Verificación:** Intento de encontrar bugs, ejecutando un programa en un ambiente simulado o de testeo. Es el proceso que provee la corrección del programa
 - **Corrección:** Un producto es correcto si posee una sintaxis adecuada. Indica si se está desarrollando el producto correctamente.
- **Validación:** Intento de encontrar bugs, ejecutando un programa en un ambiente real. Es el proceso que provee la validez del programa.
 - **Validez:** Un producto es válido si responde a una semántica adecuada. Indica si se está desarrollando el producto correcto

- Listas de Verificación y ejemplos (es lo mismo que un checklist)

<https://spa.myservername.com/qa-software-testing-checklists>

Se utilizan para describir **una lista de elementos que deben ser revisados, evaluados o completados durante el proceso de testing** para garantizar que se cumplan los estándares y requisitos de calidad.

¿Por qué necesitamos listas de verificación? : Para rastrear y evaluar la finalización (o no finalización). Tomar nota de las tareas, para que no se pase nada por alto.

Lista de Verificación de QA para Desarrollo de Software:

1. **Requisitos:**
 - ☐ ¿Se han revisado y comprendido completamente los requisitos del cliente?
 - ☐ ¿Se han identificado y documentado los requisitos no funcionales?
2. **Diseño:**
 - ☐ ¿Se ha revisado y aprobado el diseño del sistema?
 - ☐ ¿Se han considerado los principios de diseño y las mejores prácticas?
3. **Codificación:**
 - ☐ ¿Se sigue el estándar de codificación establecido?
 - ☐ ¿Se han realizado revisiones de código y se han abordado los problemas identificados?
4. **Pruebas Unitarias:**
 - ☐ ¿Se han escrito y ejecutado pruebas unitarias para cada componente?
 - ☐ ¿Todos los casos de prueba han pasado satisfactoriamente?
5. **Integración:**
 - ☐ ¿Se ha realizado la integración de todos los componentes?
 - ☐ ¿Las interfaces entre componentes funcionan correctamente?
6. **Pruebas Funcionales:**
 - ☐ ¿Se han creado casos de prueba para las funciones principales?
 - ☐ ¿Se han ejecutado las pruebas funcionales y se han documentado los resultados?
7. **Rendimiento:**
 - ☐ ¿Se han realizado pruebas de rendimiento para evaluar la capacidad de respuesta del sistema?
 - ☐ ¿Los resultados de las pruebas de rendimiento cumplen con los criterios definidos?

Especificación de Requisitos

La especificación de requisitos es un documento detallado que describe las funciones, características y restricciones que deben ser incluidas en un sistema de software. Este documento es esencial en el proceso de desarrollo de software, ya que sirve como guía para los desarrolladores, probadores y demás miembros del equipo, asegurando que todos tengan una comprensión clara y común de lo que se espera que el software logre.

Ejemplos

La especificación de requisitos típicamente incluye la siguiente información:

- **Objetivo del Sistema**
- **Requisitos Funcionales**
- **Requisitos No Funcionales**
- **Requisitos de Interfaz**
- **Requisitos de Desempeño**
- **Restricciones Técnicas**
- **Requisitos de Seguridad**
- **Casos de Uso**
- **Requisitos de Mantenimiento y Soporte**

- Cubrimientos por sentencia y por condición

La cobertura de software **es una medida que se utiliza para evaluar qué parte del código fuente de un programa ha sido ejecutada durante las pruebas.** Se utiliza para medir la efectividad de las pruebas y para identificar áreas no cubiertas, lo que puede ayudar a mejorar la calidad del software, alta cobertura no garantiza la ausencia de errores. Dos métricas comunes de cobertura son la cobertura por sentencia y la cobertura por condición. Aquí están las definiciones de ambas:

Sentencias:

- Este criterio es **necesario pero no suficiente**
- Esta cobertura requiere que se ejecute por lo menos una vez cada sentencia del programa. (cada IF/Elif)

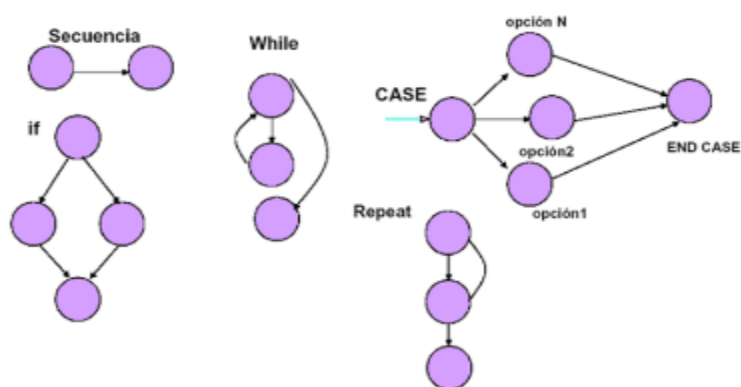
Condiciones

- Este criterio es **más fuerte que el anterior.**
- En este criterio es necesario presentar un número suficiente de casos de prueba de modo que cada condición en una decisión tenga, **al menos una vez, todos los resultados posibles.**

Árbol de complejidad ciclomática

útil porque proporciona una medida cuantitativa de la complejidad del código. Se utiliza para identificar áreas del código que pueden ser propensas a errores y para guiar las pruebas, ya que sugiere cuántas pruebas diferentes son necesarias para cubrir todas las posibles rutas a través del código.

mostrando nodos para las decisiones y arcos para las transiciones entre nodos. En este contexto, podría considerarse una estructura similar a un árbol en el sentido de que tiene nodos (decisiones) conectados por arcos (transiciones).



- Pruebas de regresión

- Son la ejecución de un subconjunto de casos de prueba, previamente ejecutados, para asegurar que las correcciones a un programa o sistema no lo degraden.
- Uno de los desafíos es la selección de los casos de prueba que se deben volver a ejecutar.
- Es el test más común en la etapa de mantenimiento de un sistema.
- Las pruebas de regresión son siempre una parte integrante de las pruebas dinámicas progresivas.
- Es el proceso de probar un producto de software después de que se hayan realizado cambios para garantizar que no se hayan introducido nuevos errores como resultado de los cambios.

Para qué sirven los alfa y beta test, y cuales son los inconvenientes que tiene el alfa test.

Alfa Test:

El Alfa Test es una fase de pruebas que ocurre antes del lanzamiento oficial de un producto de software. Durante esta fase, **el software es probado internamente por el equipo de desarrollo, y en algunos casos, por usuarios seleccionados dentro de la organización que desarrolla el software.** El **objetivo principal del Alfa Test es identificar y corregir problemas antes de que el software sea lanzado a un grupo más amplio de usuarios.**

Inconvenientes del Alfa Test:

- **Limitación de Diversidad de Usuarios**
- **Sesgo del Desarrollador**
- **Falta de Representación del Público Objetivo Completo**
- **Menos Probabilidades de Identificar Problemas Críticos**

Beta Test:

El beta testing es una fase específica de pruebas de software en la que **una versión casi completa del producto se pone a disposición de un grupo seleccionado de usuarios externos.** El propósito principal del beta testing es **recopilar comentarios del usuario real, identificar problemas no detectados internamente y evaluar el rendimiento del software en un entorno más diverso.** Estos usuarios son a menudo clientes potenciales o representantes del público objetivo.

Beneficios:

- **Identificación de Problemas del Mundo Real:**
- **Recopilación de Comentarios de Usuarios Reales:**
- **Mejora de la Satisfacción del Usuario:**
- **Pruebas de Escala y Rendimiento:**

Costos/Dificultades:

Coordinación y Gestión: Coordinar y gestionar un programa de beta testing puede requerir esfuerzos significativos. Se necesita planificación para seleccionar y comunicarse con los beta testers, distribuir versiones de prueba y recopilar y analizar comentarios.

Calidad Variable de los Comentarios: La calidad de los comentarios de los beta testers puede variar. Algunos usuarios pueden proporcionar comentarios detallados y útiles, mientras que otros pueden no dedicar el tiempo necesario para ofrecer información valiosa.

Manejo de Problemas de Seguridad y Privacidad: Asegurarse de que los datos de los usuarios estén protegidos y gestionar posibles problemas de seguridad y privacidad durante el beta testing puede requerir esfuerzos adicionales.

Desafíos Logísticos: Distribuir versiones beta a usuarios en ubicaciones geográficas diversas y gestionar la retroalimentación puede presentar desafíos logísticos. Esto era más notorio en los inicios de internet, ahora es mucho más fácil acceder a betas abiertas o por invitación.

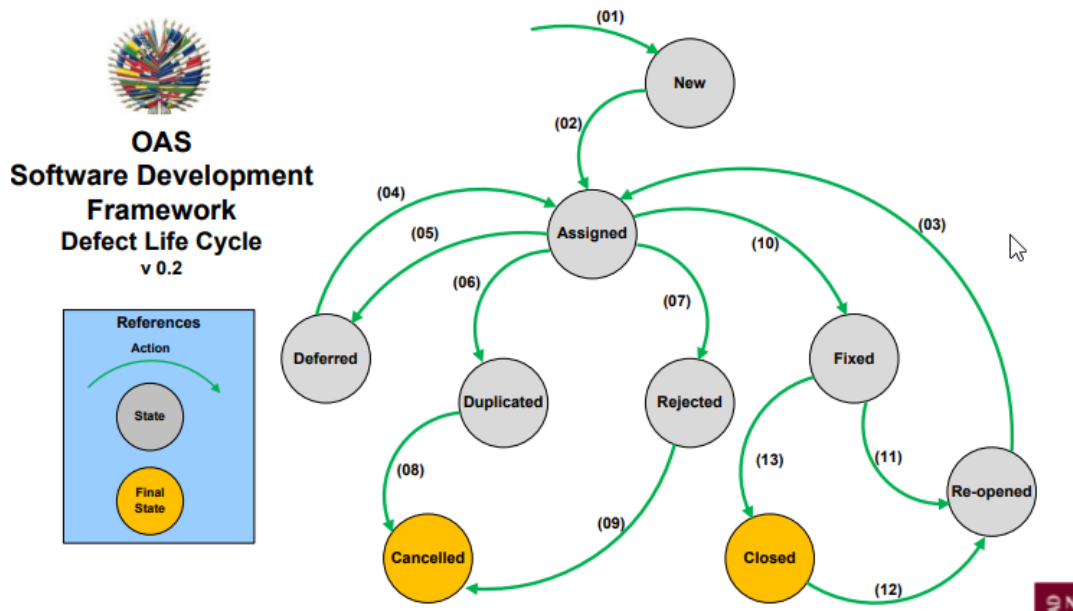
Ciclo de vida del defecto

(para qué sirve, por qué se usa)

El ciclo de vida de un defecto es el proceso mediante el cual estos defectos **se identifican, se informan, se gestionan y finalmente se resuelven**. Sirve para realizar una correcta gestión de incidencias. Gracias a ello, podemos implantar metodologías de mejora continua en nuestra organización que nos permitan optimizar nuestros tiempos y resultados.

Estados del ciclo de vida del defecto:

- **Nuevo:** Defecto que se acaba de crear y aún no se ha validado.
- **Asignado:** Defecto que es asignado a un equipo de desarrollo o desarrollador para hacer frente a él. Aún no está resuelto.
- **Activo:** El defecto está siendo controlado por integrante del equipo y la investigación está en curso. En esta etapa hay dos resultados posibles: aplazado o rechazado.
- **Para probar:** El defecto está resuelto por el equipo de desarrollo y listo para probarlo.
- **Verificado:** El defecto se prueba y la prueba ha sido llevada a cabo por el equipo de calidad.
- **Cerrado:** El estado final del defecto puede ser cerrado después de las pruebas por parte del equipo de calidad. También se puede cerrar un defecto está duplicado o considerado como que no es un defecto.
- **Reabierto:** Cuando el equipo de desarrollo arregla el defecto pero el equipo de calidad lo prueba y comprueba que aún no está resuelto. En este caso se vuelve a abrir el mismo defecto pero con un estado de reabierto.
- **Aplazado:** A este estado llegamos cuando un defecto no se puede abordar en este sprint o versión y se pasa al siguiente sprint o versión.
- **Rechazado:** Un defecto puede ser rechazado por 3 razones: está duplicado, no es un defecto o no es posible reproducirlo.



- Métricas para qué sirve y decir tipos de métricas

Id	Descripción	Tipo
CCPE	Cantidad de casos de prueba ejecutados	Cuantitativo
PCPE	Porcentaje de casos de prueba ejecutados.	Cuantitativo
CDE	Cantidad de defectos encontrados.	Cuantitativo
PDES	Porcentaje de defectos de encontrados por severidad.	Cuantitativo
PCS	Porcentaje de cubrimiento de sentencias.	Cuantitativo
PCC	Porcentaje de cubrimiento de condiciones.	Cuantitativo

El objetivo principal de las métricas de control de calidad es determinar si las técnicas de prueba actuales ayudan a alcanzar el nivel de calidad deseado. También son útiles para identificar los obstáculos y los problemas de productividad en el proceso de prueba, lo que resulta en lanzamientos de software más rápidos.

Smoke, para que se usa, un ejemplo

Las pruebas de humo (smoke testing en inglés) son un tipo de pruebas funcionales que consisten en una revisión rápida de un producto de software para comprobar su inicio y que no tiene defectos evidentes que interrumpan la operación básica del mismo. El nombre es por analogía a las pruebas rudimentarias en ingeniería electrónica, en las que se comprueba que el encendido de un circuito no causa humo ni chispas.

Las pruebas de humo formales se llevan a cabo en momentos importantes del proceso de creación del software, por ejemplo, antes de realizar las pruebas funcionales de las nuevas características.

Qué son las pruebas estáticas

Proceso de revisar o analizar un producto de software con el objeto de identificar defectos y mejoras. **No hay ejecuciones de prueba ni código. tienen como objetivo identificar problemas como información repetida, ambigüedad o contradicciones en la documentación**

Fases

- Planificación
- Orientación inicial
- Preparación individual
- Reunión de revisión
- Seguimiento
- Evaluación

Fases de la pruebas dinámicas y definir preparación

- **Planificación**
 - **Diseño**
 - **Derivación de casos**
 - **Preparación**
 - **Ejecución de cada caso de prueba**
 - **Análisis y evaluación**
-
- **Planificación:** Se define el plan de pruebas y los objetivos de la misma. Se debe especificar, entre otros:
 - Funciones, procesos y características a testear y a no testear.
 - Métodos, tipos y orden del testeo
 - Criterios de aprobación de la prueba.
 - Criterio de clasificación de severidad de bugs.
 - Ambientes, recursos físicos y herramientas a utilizar.
 - Recursos humanos, responsabilidades y cronograma.

- **Diseño:** Se determina cómo cumplir con los objetivos establecidos en la planificación. Se debe especificar, entre otros:
 - Condiciones generales a testear e ítems generales a verificar por función o proceso.
 - Procedimientos de ejecución de casos de prueba.
- **Derivación de casos:** Es la especificación de cada uno de los casos de prueba. Para cada caso se debe especificar, entre otros:
 - Identificador.
 - Función a testear.
 - Condiciones a testear.
 - Resultado esperado
- **Preparación:** Es la confección de todos los elementos necesarios para la ejecución de la prueba, tales como:
 - Archivos
 - Scripts
 - Formulario
 - Tarjetas.
- **Ejecución de cada caso de prueba.** se debe registrar, entre otros:
 - Resultado obtenido.
 - Fecha, hora y responsable de la ejecución.
- **Análisis y evaluación** Se analizan los resultados obtenidos y en base a los criterios establecidos en la planificación. Se debe determinar, entre otros
 - Condiciones de suceso de cada bug detectado
 - Severidad de cada bug
 - Aprobación o no de la prueba